

Documento de Análise de Portabilidade - Prótese Flow

1. Introdução

A portabilidade é a medida da facilidade com que um software pode ser transferido de um ambiente de execução para outro, exigindo poucas ou nenhuma modificação ao ser movido. Este documento apresenta a análise completa da portabilidade do sistema Prótese Flow, detalhando a transição do ambiente de desenvolvimento local para um ambiente de produção em nuvem. O objetivo é avaliar a viabilidade, mitigar problemas de configurações e criar um histórico documentado para futuras implementações.

2. Identificação dos Ambientes

- Ambiente Original (Desenvolvimento):
 - Sistema Operacional: Windows/Linux (Localhost).
 - Base de Dados: SQLite.
 - Armazenamento: Local (File System) para arquivos CAD.
- Ambiente de Destino (Produção / Cloud):
 - Sistema Operacional: Distro Linux em ambiente Cloud.
 - Base de Dados: PostgreSQL.
 - Armazenamento: Object Storage na Nuvem.

3. Questionário de Avaliação de Portabilidade

Para identificar e quantificar as dependências do software em relação ao ambiente original, o seguinte questionário foi respondido com base na arquitetura do Protese Flow:

1. O software depende fortemente de hardware específico, como placas de vídeo de alto desempenho, processadores com instruções especiais, ou periféricos dedicados?

Não. O Protese Flow é uma aplicação web. O processamento pesado (renderização do CAD) ocorre fora do sistema, nos softwares de modelagem dos cadistas. O servidor precisa apenas de processamento padrão para requisições HTTP e

manipulação de arquivos.

2. O sistema exige uma versão específica de um sistema operacional ou recursos nativos exclusivos dele?

Não. O sistema funciona em qualquer um dos sistemas operacionais. Por utilizar Python e JavaScript/Node, o código roda de forma idêntica em Windows, macOS ou distribuições Linux.

3. O código-fonte faz uso de bibliotecas de terceiros que não estão disponíveis ou não são suportadas em outros ambientes?

Todas as bibliotecas utilizadas são mantidas via pip e possuem suporte amplo em qualquer ambiente que rode Python, então ele já contém todos os recursos necessários.

4. Quais são as linguagens de programação e frameworks principais utilizados no projeto? Eles são multiplataforma?

O backend utiliza Python com o framework Django. O frontend pode utilizar JavaScript/TypeScript com bibliotecas modernas (React/Tailwind). Ambos os ecossistemas são nativamente multiplataforma.

5. O projeto utiliza sistemas de gerenciamento de banco de dados (SGBDs) que impõem restrições de ambiente ou que possuem dialetos SQL altamente específicos?

Embora o ambiente local utilize SQLite e a produção utilize PostgreSQL, o mapeamento objeto-relacional do Django não considera os dialetos SQL. Isso garante que a troca de banco de dados seja feita apenas alterando credenciais, sem reescrever consultas SQL.

6. Existe dependência de serviços externos ou APIs que podem ter disponibilidade ou comportamento diferente em outros ambientes de execução?

Sim, existe a dependência futura de um serviço de Cloud Storage (como AWS S3) para armazenar os arquivos `.STL` e `.PLY`. Contudo, essa integração é feita via

bibliotecas portáteis ([django-storages](#)), garantindo que o comportamento seja previsível em qualquer ambiente.

7. O sistema foi projetado com configurações "hardcoded" (fixas no código) que dificultam a adaptação para caminhos de arquivo, URLs de serviços ou configurações de rede diferentes?

Não. Seguindo as boas práticas de portabilidade, dados sensíveis, URLs de banco de dados e chaves secretas não estão "hardcoded". São gerenciados através de variáveis de ambiente (arquivos [.env](#)).

8. A interface do usuário é construída de forma responsiva e adaptável a diferentes tamanhos de tela e resoluções?

Sim. O frontend utiliza padrões responsivos garantindo a usabilidade tanto em monitores ultrawide de cadistas quanto em tablets ou celulares de dentistas.

9. Existem testes automatizados que cobrem as principais funcionalidades do sistema e podem ser executados no novo ambiente para garantir que não houve regressões?

O repositório base conta com a estrutura nativa de testes do Django (*tests.py* nos módulos). Estes testes servem como garantia para validar se as lógicas de mudança de estado do pedido (Pendente > Em Produção) continuam a funcionar perfeitamente após a transição de ambiente.

10. A documentação técnica detalha claramente os requisitos do ambiente de execução e as etapas necessárias para a configuração em um novo ambiente?

Sim. O documento atualiza e consolida essas informações, complementado pelo arquivo *requirements.txt* do projeto, que dita as dependências exatas para a configuração do novo ambiente de forma automatizada.

4. Plano de Implementação

A portabilidade do sistema ocorreu através do seguinte fluxo prático:

1. Versionamento: Utilização do Git para centralizar o código base.

2. Separação de Variáveis: Substituição de caminhos e credenciais locais por variáveis de ambiente (*.env*).
3. Migração do Banco de Dados: Configuração das variáveis do PostgreSQL e execução do comando *python manage.py migrate* no ambiente de destino.
4. Resolução de Dependências: Instalação das bibliotecas no servidor cloud via *pip install -r requirements.txt*.

5. Conclusão Final e Recomendações

O processo de portabilidade foi avaliado pela gestão do projeto é considerado bem-sucedido e viável. O desacoplamento de configurações e o uso de linguagens multiplataforma reduziram drasticamente os riscos.

Para manter a portabilidade facilitada e reter o *know-how* técnico em futuras implementações, recomenda-se:

- Manter o uso de padrões abertos: Comunicação via APIs REST.
- Evitar APIs de sistema: Não utilizar recursos exclusivos do Windows ou Linux diretamente no código.
- Adoção de Docker: Para futuras evoluções, isolar a aplicação em contentores para garantir que o software funcione de forma idêntica independentemente do servidor físico.